HERMES · PHASE 1 · LESSON 0006 · LAB EXERCISE 05-MODEL-GATEWAY · ~45–60 MIN

The Model Gateway

Exercise 04's client imports the SDK, catches its errors, and keeps the job's ledger in one file. Where does the line between the SDK and Hermes go?

Why this matters for Hermes

Scenario step S4 (model calls) puts every one behind the *ModelGateway*: Hermes' one interface to any model provider. SDK mechanics run below it, and Hermes policy sits above it. Failures reach the supervisor as data to classify. It also carries the abort signal S6 (budget limits) needs, and the request id for S7 (the trace). Two threads advance: the six responsibilities move below the line, and lesson 0005's parse lands.

1 · The cost of copying the client

Open exercise 02's `client.ts`. One file builds the client, catches three error classes, and reads `message.usage` into a ledger. Lessons 0008 to 0011 would repeat all of it in every file that wants a model call.

- A 429 unwinds the caller's stack. Every caller writes the same `instanceof` chain to survive one.
- Policy tests need the mock process running, and a stale one has cost three sessions.
- Code that names `Anthropic.Message` couples to the provider at every call site.

One interface fixes all three.

Layer	Changed?	Evidence
the wire	no	Part A sends exercise 02's request unchanged
the six responsibilities ①–⑥	no	endpoint, auth, version, shape, errors, cancellation. The SDK still runs all six, below the port
the types the domain sees	yes	`supervisor.ts` imports `gateway.ts`, nothing else
the failure surface	yes	the supervisor switches on `failure.kind`, no `try/catch`
testability	yes	Part C runs the supervisor in 0.6 ms, mock stopped
the import direction	yes	the adapter imports the port, the domain no adapter

New today.

Five ideas are new.

- A port is an interface the domain owns.
- An adapter translates calls down, replies up, and exceptions into data.
- Above the port, every outcome of a call is a value.
- A fake is a working implementation of the port, running in memory.
- Provider neutrality is a property of the interface, not a count of adapters.

Reprise: the six responsibilities, typed error classes, the abort signal, `safeParse`, and narrowing a union on its tag.

2 · The port: an interface the domain owns

You have met this move at the wire. Lesson 0002's base URL swapped the real API for the mock. A *port* is an interface the caller owns, written in the caller's own words. It swaps implementations at build time instead. One word now carries two meanings.

The word	What it names	Where you met it
port `8787`	a TCP address, where a process listens for bytes	the mock, since exercise 01
the port in ports and adapters	an interface the domain owns, where implementations plug in	`gateway.ts`, built today

Read the declaration first, as lesson 0003 read the SDK's entry point (`05-model-gateway/src/gateway.ts`, abridged).

```
export interface ModelCall  { prompt: string; maxTokens: number }
export interface ModelReply { text: string; stop: StopCause;
                              usage: { inputTokens: number; outputTokens: number };
                              requestId: string | null }

export type GatewayResult =                       // narrow on `ok` — lesson 0005's move
  | { ok: true;  reply: ModelReply }
  | { ok: false; failure: GatewayFailure };

export interface ModelGateway {
  complete(call: ModelCall, options?: { signal?: AbortSignal }): Promise<GatewayResult>;
}
```

Syntax	Meaning	At runtime?
`interface ModelGateway`	an interface in the domain's words	nothing. TypeScript erases every interface
`implements ModelGateway`	a build-time claim on a class	nothing of the claim. The class survives
`gateway.complete(call)`	a call through the port's signature	whichever implementation the wiring chose
`options?.signal`	the supervisor's cancellation signal crosses	a real `AbortSignal`, owned by the web platform

A signature tells TypeScript what may be called. The implementation does the runtime work. One signature has two implementations today, and the domain cannot tell which one it holds.

Dependency inversion names the arrow that just turned around. The concrete code imports the domain's interface, and the domain imports no concrete class. One command checks it. Run `grep -r "@anthropic-ai/sdk" src/` and you get the adapter and the wiring.

```
// Part A — the proof arm                  // Part B1 — the refusal arm
{                                          {
  ok: true,                                  ok: false,
  reply: {                                   failure: {
    text: "Hello from the mock. These bytes…",              kind: "throttled",
    stop: "completed",                         retryAfterMs: 5000,
    usage: { inputTokens: 11,                },
             outputTokens: 42 },           }
    requestId: "req_mock_0001",            // the wire said `retry-after: 5` (seconds).
  },                                       // the port speaks milliseconds: translation,
}                                          // not passthrough
```

Three spellings change at the port. `end_turn` became `completed`, and `input_tokens` became `inputTokens`. A `RateLimitError` became `{ kind: "throttled" }`. No provider spelling survives above the line.

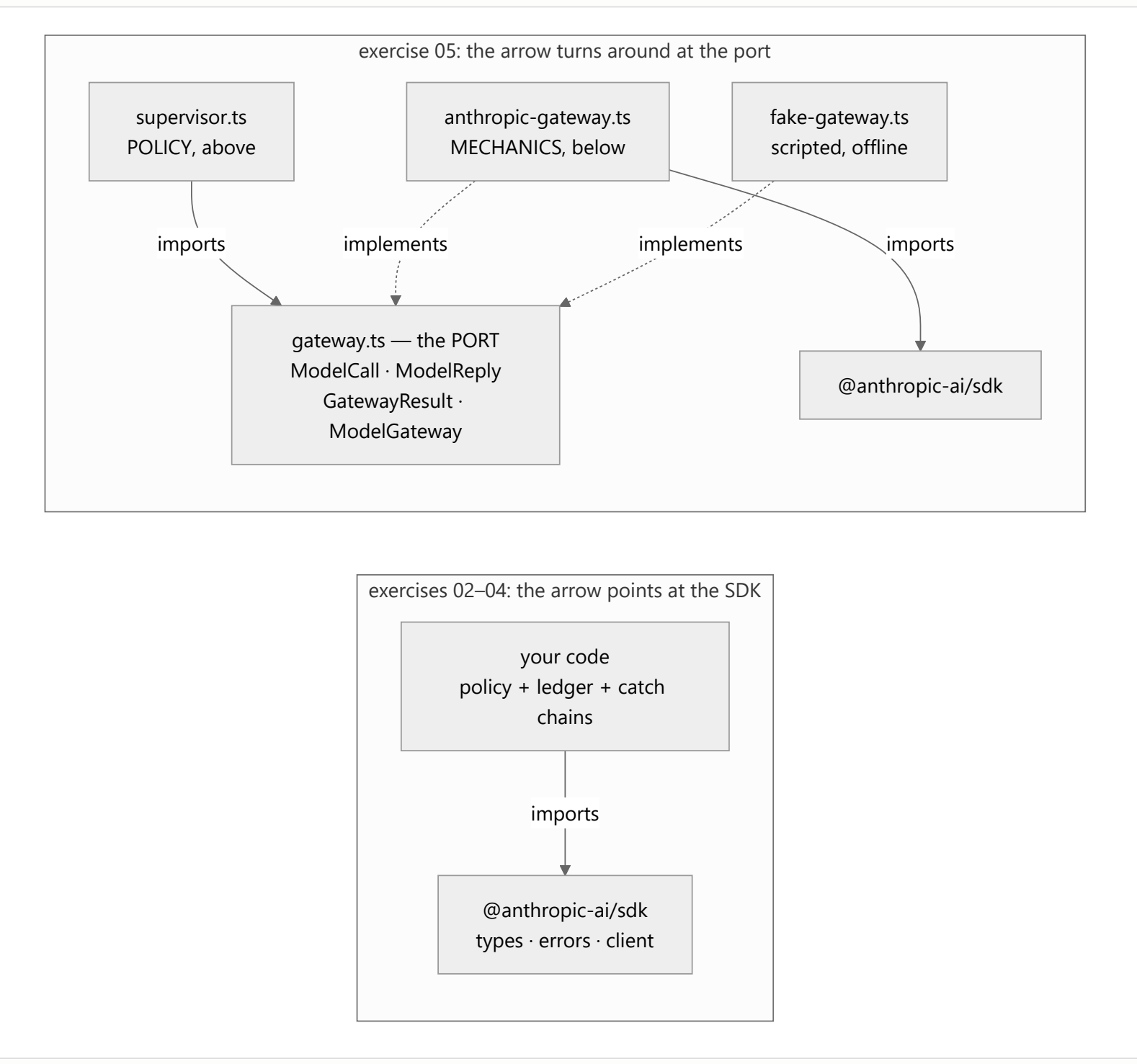

Diagram 1. Dependency inversion, counted in import statements. Both implementations depend on the domain's own interface.

3 · The adapter: three translations, no new bytes

During Part A the mock's terminal prints the exchange exercise 02 produced.

```
POST /v1/messages  (req_mock_0001)
x-api-key: … · anthropic-version: 2023-06-01 · content-type: application/json
{ "model": "claude-opus-4-8", "max_tokens": 100, "messages": [ … ] }

HTTP/1.1 200 OK · request-id: req_mock_0001 · stop_reason=end_turn input_tokens=11
```

The port added nothing to it. The adapter sends no extra header and no second request. TypeScript erases the port before the adapter builds this request. An *adapter* is a class that implements a port by translating one concrete dependency into the port's words. This one translates three things.

1. The call goes down. `ModelCall` becomes the SDK's params object, and the model id arrives as constructor configuration.
2. The reply comes up through the boundary parse. `WireReplySchema.safeParse` checks the fields `ModelReply` consumes.
3. The failures come up as data. The adapter writes exercise 02's `instanceof` chain one last time, and returns a `GatewayFailure` value.

SDK error classes are runtime values, which is why `instanceof` works on them. The port's interfaces are erased. The adapter rethrows any error it cannot classify, because a bug must reach the caller.

```
B1 throttled → { outcome: 'retry_later', tokensSpent: 0,
                 notes: ['provider asked for 5000 ms of backoff'] }
B2 aborted   → { outcome: 'gave_up',     tokensSpent: 0, notes: ['aborted by the operator'] }
B3 malformed → { outcome: 'gave_up',     tokensSpent: 0, notes: [
                 'reply refused at the boundary',
                 'stop_reason: [invalid_value] Invalid option: expected one of …',
                 'usage.output_tokens: [invalid_type] Invalid input: expected number, received string' ] }
```

B3 sends exercise 04's drifted bytes. They once produced a ledger of `"1142"` against a real spend of 53. Here they never reach a ledger. A plausible silent error is more dangerous than a conspicuous false alarm. You made that argument in lesson 0005.

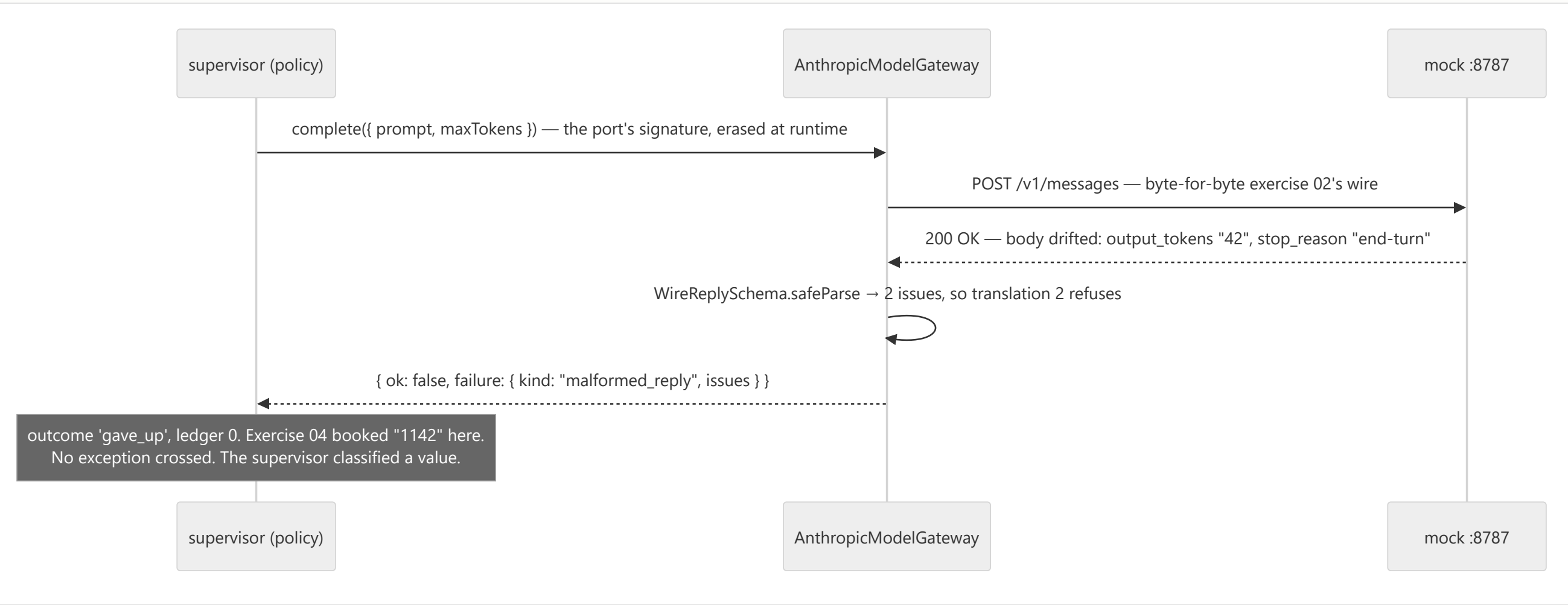
supervisor (policy)
AnthropicModelGateway
mock :8787
complete({ prompt, maxTokens }) — the port's signature, erased at runtime
POST /v1/messages — byte-for-byte exercise 02's wire
200 OK — body drifted: output_tokens "42", stop_reason "end-turn"
WireReplySchema.safeParse → 2 issues, so translation 2 refuses
{ ok: false, failure: { kind: "malformed_reply", issues } }
outcome 'gave_up', ledger 0. Exercise 04 booked "1142" here.
No exception crossed. The supervisor classified a value.
supervisor (policy)
AnthropicModelGateway
mock :8787

Diagram 2. Part B3, measured. The drifted 200 travels the unchanged wire, the adapter refuses it, and policy receives data.

4 · The fake, and the provider decision

A *fake* is a working implementation of a port that answers from a script. `FakeModelGateway` is the port's second implementation, and a test artifact rather than a second provider.

	the mock server	`FakeModelGateway`
doubles the API at	the wire: real HTTP on port 8787	the port: a method call, in memory
exercises	transport, the six responsibilities, the parse	the supervisor's decisions alone
costs, measured	~400 ms per call, latency on purpose	0.6 ms for two calls, mock stopped

Part C stops the mock and runs the same supervisor against the fake. The landed report matches Part A's, except that `request` reads `(none)`. The scripted throttle drives the `retry_later` branch B1 drove over the wire.

Hermes ships a provider-neutral port with one live adapter. That question had stood open since session 1. The port's words name text, a stop cause, two usage numbers and a failure kind, so neutrality cost nothing. No benchmark asks for a second live adapter, and MISSION rules out speculative work. Provider choice arrives later, above the port, as a routing decision.

A port built against one provider can copy that provider's vocabulary. The fake answers that risk from day one, and proves nothing about a real second provider.

Capability	Status	Where it lives, or when it arrives
Model calls cross a port	implemented	`gateway.ts`, `anthropic-gateway.ts`, Part A
An abort signal crosses it	prepared	Part B2 proves it. Budget rules arrive in lesson 0009
The request id survives	prepared	The adapter keeps it. The trace is lesson 0011
A rule picks the model	planned	No `ROADMAP.md` row names one yet

5 · The exercise

Commands and the measured table live in `05-model-gateway/README.md`. The arc:

1. Read `gateway.ts`, `supervisor.ts`, then `anthropic-gateway.ts`. At each import, say which side of the port you are on.
2. Run Part A and watch both terminals: the supervisor's report, and the unchanged wire.
3. Run `pnpm job b` first. A raw `RateLimitError` crashes the job, and that crash is what you are replacing.
4. Implement `classifyFailure`, most specific case first. Rethrow what you cannot classify, then re-run for three failures and no crash.
5. Implement `FakeModelGateway.complete`, stop the mock, and run `pnpm job c`. Compare C2's report with B1's.

6 · Exercise: mechanics or policy?

Place each item on its side of the port

Answer from memory before you click. Feedback is immediate.

A throttled job is parked and retried after the provider's backoff window.

policy, above the port | mechanics, below the port | the port itself | the wire, untouched

A 429 is retried automatically while honoring the `retry-after` header.

policy, above the port | mechanics, below the port | the port itself | the wire, untouched

The `GatewayResult` union: proof on one arm, classified failure on the other.

policy, above the port | mechanics, below the port | the port itself | the wire, untouched

An `instanceof` chain turns `RateLimitError` into `{ kind: "throttled" }` data.

policy, above the port | mechanics, below the port | the port itself | the wire, untouched

`POST /v1/messages` with gates ② auth, ③ version, ④ the request shape.

policy, above the port | mechanics, below the port | the port itself | the wire, untouched

Which model id serves this job.

policy, above the port | mechanics, below the port | the port itself | the wire, untouched

Score: 0 / 0

7 · Quiz

Four questions

Q1. Exercise 05 ships `interface ModelGateway` with two implementations. What exists of the interface at runtime?

- A registry object that maps each adapter onto its port methods
- Nothing. It is erased, and the type-checker enforced it earlier
- A shared prototype that both adapter classes silently inherit from
- A wrapper function that routes each call to the chosen adapter

Q2. The SDK already declares `Message` as `create()`'s return type. Why does the adapter still run `safeParse`?

- Because the SDK validates the headers only, and bodies need their own pass
- Because Zod repairs any drifted fields that the SDK would leave set to null
- Because the declared type is an erased claim, so the parse is the real check
- Because safeParse converts the wire's snake_case into Hermes's camelCase

Q3. Exercise 01's mock server and Part C's `FakeModelGateway` are both test doubles. What separates them?

- The mock doubles the wire as a process, and the fake doubles the port
- The mock validates each request strictly, and the fake accepts anything
- The mock replays recorded fixtures, and the fake computes each reply
- The mock serves raw-fetch clients, and the fake serves SDK-based ones

Q4. Which statement records the provider-neutrality decision as this lesson landed it?

- Two live adapters ship today, so the port is proven against real providers
- The port stays Claude-shaped until Phase 4 benchmarks justify abstracting
- Neutral port, one live adapter. Neutrality lives in the interface itself
- The decision is deferred again, and the gateway re-exports the SDK's types

Score: 0 / 0

8 · Say it in your own words

Answer these in the chat after the lab, with no notes.

1. Define port and adapter in one sentence each. Say which one survives compilation, and what enforces the one that does not.
2. Trace a 429 from the wire to the supervisor. Name what the SDK hands the adapter, and what the adapter hands the port.
3. A reviewer calls the adapter's `safeParse` redundant, because the SDK types the reply. Answer in two sentences.

If you keep only five sentences from Phase 1's opening, keep these. A port is an interface the domain owns, and the domain and its adapters both depend on it. The adapter translates calls down, replies up through the parse, and exceptions into failure data. Above the port every outcome is a value the supervisor classifies. A fake implements the port in memory, so policy is testable with no server.

Primary source

Read Alistair Cockburn's Hexagonal Architecture note, two pages from the pattern's author. Then read Martin Fowler's Test Double taxonomy, the source of our mock and our fake.

Version pin. Every excerpt and number here was verified against `@anthropic-ai/sdk` 0.113.0 and `zod` 4.4.3, by live runs against the exercise 01 mock. The four new terms have notes under `wiki/terms/`.

Two supplements sit beside this lesson. What Counts as Policy defines the word it leans on. The Hermes Control Plane says what a Hermes job is. Read section 1, then 0006b, then 0006a.

Phase 1 is open. Next comes lesson 0007, *The TaskSpec Is a Contract*, with lab `06-taskspec`. Lesson 0005's parse meets S1 (the envelope check) there, and Hermes rejects invalid work before it spends a token.